

Desenvolvimento Web 2

Aula 3

Orientação a Objetos em JavaScript **Tipos Primitivos e de Referência**

`l.bertholdo@ifsp.edu.br`

Conteúdo

- Tipos Primitivos
 - Identificando Tipos Primitivos
 - Métodos de Tipos Primitivos
- Tipos de Referência
 - Criando Objetos
 - Removendo Referências a Objetos
 - Adicionando Propriedades
 - Instanciando Tipos Próprios
 - Formas Literais
 - Acesso a Propriedades
 - Identificando Tipos de Referência
 - Tipos Wrapper Primitivos
- Classes

Tipos Primitivos e de Referência

- Os **objetos** constituem a parte central da linguagem JavaScript, pois quase todos os dados são representados por meio de objetos.
- Até mesmo **funções** são representadas como objetos em JavaScript, o que as tornam **funções de primeira-classe**.

***Funções first-class** são funções tratadas como uma variável. Isso faz com que a função possa ser atribuída como um valor à uma variável, possa ser passada como argumento para outra função e, até mesmo, ser retornada por outra função.*

- Em JavaScript, é possível criar objetos a qualquer momento, e adicionar ou remover propriedades deles sempre que necessário.
- Mas antes de estudar os objetos, é preciso entender os dois principais grupos de tipos de dados da JavaScript: os **tipos primitivos** e os **tipos de referência**, ambos representados por meio de objetos.

Tipos Primitivos e de Referência

- Os **tipos primitivos** são armazenados como tipos de dados simples. Já os **tipos de referência** são armazenados como objetos que, na realidade, são apenas referências a posições de memória.
- **Valores primitivos** são armazenados diretamente em um objeto variável, enquanto os **valores de referência** são criados como ponteiros dentro de um objeto variável.
- Uma das principais características da linguagem JavaScript é que ela permite tratar **tipos primitivos** como **tipos de referência**, o que é feito por meio de um **objeto variável**.

*É importante dizer que **valores primitivos** não são objetos de fato. A JavaScript apenas faz com que eles pareçam ser objetos para oferecer **maior uniformidade à linguagem**, ou seja, para fazer com que todos os seus dados e funções sejam tratados como objetos.*

Tipos Primitivos

- Todos os tipos primitivos têm **representações literais** de seus valores. Um **literal** representa um valor em si (não armazenado em uma variável). Os principais tipos primitivos para representar **dados simples** são:

Tipo	Descrição	Literal (parte direita da igualdade)
boolean	true ou false.	<code>var aprovado = true;</code>
number	Qualquer valor numérico inteiro ou de ponto flutuante.	<code>var idade = 25;</code> <code>var altura = 1.72;</code>
string	Um caractere ou uma sequência de caracteres delimitados por aspas simples ou duplas.	<code>var nome = "Pedro";</code> <code>var letra = "a";</code>
null	Tipo primitivo que tem apenas o valor null (atribuído a uma variável que foi inicializada, mas não contém um valor).	<code>var objeto = null;</code>
undefined	Tipo primitivo que tem apenas o valor undefined (atribuído a uma variável que ainda não foi inicializada).	<code>var flag = undefined;</code> <code>var ref; // undefined</code>

Comportamento dos Tipos Primitivos

Quando o **valor primitivo** “azul” é atribuído à variável **cor1**, ele é copiado para esta variável. Ao atribuir esta variável à variável **cor2**, cada variável terá a sua própria cópia do dado. Logo, mudar o valor de uma variável não afeta a outra, já que estão em locais de memória diferentes.

```
var cor1 = "azul";
var cor2 = cor1;
console.log(cor1); // "azul"
console.log(cor2); // "azul"
cor1 = "verde";
console.log(cor1); // "verde"
console.log(cor2); // "azul"
```

Identificando Tipos Primitivos

- Para identificar tipos primitivos, pode-se usar o operador **typeof**, que retorna uma string indicando o tipo de dado.

```
console.log(typeof "Pedro");  
console.log(typeof 25);  
console.log(typeof 1.51);  
console.log(typeof true);  
console.log(typeof undefined);  
console.log(typeof null);
```

string
number
number
boolean
undefined
object

```
console.log(valor === null); // true ou false
```

Note que o valor **null** retorna “object”, dando a entender que **null** é uma referência a um objeto vazio (nulo). Se o objetivo for verificar se um valor é nulo, a melhor forma é compará-lo diretamente com **null**, usando o operador de igualdade de tipo e valor (**===**) ou diferença de tipo ou valor (**!==**).

Métodos de Tipos Primitivos

- Apesar de não serem objetos, os tipos primitivos **string**, **number** e **boolean** contêm **métodos**. O tipo **string**, em particular, tem vários métodos para manipular cadeias de caracteres. Exemplos:

```
var nome = "Pedro";  
var minusculas = nome.toLowerCase();  
var primeiraLetra = nome.charAt(0);  
var letrasDoMeio = nome.substring(1, 4);  
var numero = 10;  
var duasCasasDecimais = numero.toFixed(2);  
var aprovado = true;  
var aprovadoString = aprovado.toString();  
  
console.log(nome);  
console.log(minusculas);  
console.log(primeiraLetra);  
console.log(letrasDoMeio);  
console.log(numero);  
console.log(duasCasasDecimais);  
console.log(aprovado);  
console.log(aprovadoString);
```

Pedro	← Retorna string
pedro	← Retorna string
P	← Retorna string
edr	← Retorna string
10	← Retorna number
10.00	← Retorna string
true	← Retorna boolean
true	← Retorna string

Tipos de Referência

- Os tipos de referência representam objetos em JavaScript e se assemelham às **instâncias de classes** das linguagens orientadas a objetos tradicionais.
- Valores de referência são **instâncias de tipos de referência**, ou seja, são **instâncias de objetos**.
- Um objeto é constituído de **propriedades**. Quando o valor de uma propriedade for uma **função**, ela pode ser chamada de **método**.

*As funções propriamente ditas também são **valores de referência (objetos)** em JavaScript. Na verdade, em JavaScript, **objetos** podem ser compostos de **propriedades de dados** (que podem ser objetos ou tipos primitivos) e de **propriedades do tipo função** (que são objetos).*

Criando Objetos

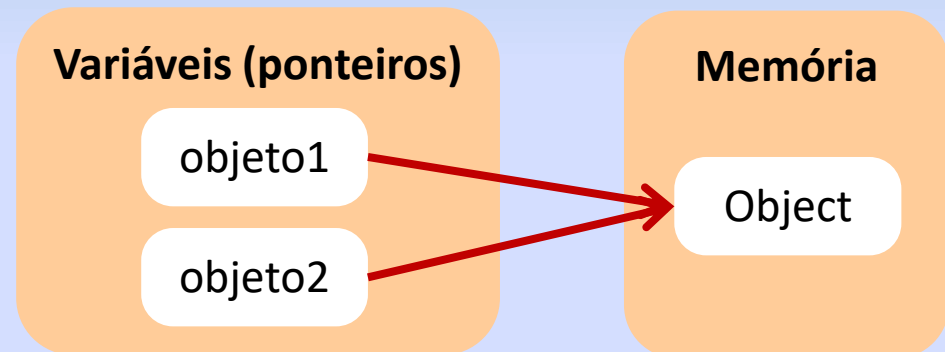
- Uma das formas de criar/instanciar um objeto é por meio do operador **new** e um **construtor** (uma função que usa **new** para criar um objeto).
- Por convenção, os nomes dos **construtores** em JavaScript iniciam com letra maiúscula para distingui-los das outras funções. Exemplo:

```
// Instancia um objeto genérico e armazena uma referência a ele na variável objeto.  
var objeto = new Object();
```

Criando Objetos

- Quando um objeto é atribuído a uma variável, na verdade, estamos atribuindo um **ponteiro** a esta variável.
- Com isso, ao atribuir uma variável a outra, cada variável terá uma cópia deste ponteiro, e ambos estarão referenciando o mesmo objeto (conteúdo) armazenado na memória.

```
var objeto1 = new Object();  
var objeto2 = objeto1;
```



Como as duas variáveis estão referenciando o mesmo objeto na memória, qualquer alteração feita em uma das variáveis será refletida na outra.

Removendo Referências a Objetos

- JavaScript possui um **coletor de lixo** (*garbage collector*) para que os desenvolvedores não precisem se preocupar com alocações de memória ao usar **tipos de referência**.
- Isso torna desnecessário, na maioria dos casos, a atribuição de **null** a um objeto.
- No entanto, o valor **null** pode ser útil para verificar se uma variável ou um objeto é nulo antes de tentar acessar suas propriedades.

```
var objeto = new Object();  
objeto.propriedade = "Olá!";  
  
if (objeto !== null)  
    console.log(objeto.propriedade);
```

Adicionando Propriedades

- Em JavaScript, é possível adicionar propriedades a um objeto a qualquer momento. Exemplo:

```
var objeto1 = new Object();  
var objeto2 = objeto1;  
objeto1.minhaPropriedade = "Olá!";  
console.log(objeto1.minhaPropriedade);  
console.log(objeto2.minhaPropriedade);
```

```
Olá!  
Olá!
```

Nesse exemplo, **minhaPropriedade** é adicionada ao **objeto1** com valor igual a "Olá!". Como o **objeto2** referencia o mesmo local de memória do **objeto1**, a propriedade fica acessível em ambos os objetos.

Instanciando Tipos Próprios

- Para criar objetos genéricos, usamos o tipo **Object**, porém a linguagem JavaScript oferece outros **tipos de referência próprios**:

Tipo	Descrição	Exemplo de Instanciação
Array	Lista ordenada de valores indexados numericamente	<code>var itens = new Array();</code>
Date	Data e hora	<code>var agora = new Date();</code>
Error	Erro de execução	<code>var erro = new Error("Erro!");</code>
Function	Função	<code>var funcao = new Function(console.log("Olá!"));</code>
RegExp	Expressão regular	<code>var busca = new RegExp("tranquilo calmo sereno");</code>

Formas Literais

- Assim como os tipos primitivos, os **tipos de referência** também têm suas **formas literais**.
- A forma literal dos tipos de referência consiste na definição de um valor de referência sem criar um objeto explicitamente (usando o operador **new** e o **construtor** do objeto).
- Para criar um **objeto** na forma literal, pode-se declarar o objeto e definir suas propriedades entre chaves. Exemplo:

Forma Literal

```
var livro = {  
    titulo: "O Nome da Rosa",  
    ano: 2011  
};
```

Forma não Literal

```
var livro = new Object();  
livro.titulo = "O Nome da Rosa";  
livro.ano = 2011;
```

Formas Literais

- Para criar um **array** na forma literal, pode-se declarar o *array* inserindo qualquer quantidade de elementos separados por vírgula dentro de colchetes. Exemplo:

Forma Literal

```
var cores = ["azul", "verde", "vermelho"];  
console.log(cores[0]); // azul
```

Forma não Literal

```
var cores = new Array("azul", "verde", "vermelho");  
console.log(cores[0]); // azul
```

- Para criar uma **função** na forma literal, basta usar a palavra reservada **function**, declarar o nome da função e definir seus parâmetros. Exemplo:

Forma Literal

```
function retorna(valor) {  
    return valor;  
}
```

Forma não Literal

```
var retorna = new Function("valor", "return valor;");
```

*Quase sempre as funções são definidas por meio de sua forma literal, sejam funções declaradas (com nome) ou anônimas. Na verdade, usar o construtor **Function** é desaconselhável por ser mais difícil manter e depurar os parâmetros da função, já que se encontram em formato string.*

Formas Literais

- Para criar uma **expressão regular** na forma literal, deve-se definir a expressão entre duas barras. Também pode-se incluir uma **opção adicional** que corresponde a uma única letra após a 2ª barra. Exemplo:

Forma Literal

```
var digitos = /\d+/g;
```

Forma não Literal

```
var digitos = new RegExp("\\d+", "g");
```

/ e /: Barras que delimitam o início e o fim da expressão regular.

\d: Busca qualquer dígito numérico de 0 a 9.

+: Busca por sequências do caractere anterior, no caso, o **\d**.

g: Opção adicional que retorna todas as ocorrências do padrão, e não apenas a primeira.

*Usar a **forma literal** das expressões regulares é mais fácil do que usar o **construtor RegExp**, porque não é necessário se preocupar com **caracteres de escape** nas strings sempre que houver uma barra invertida (\), como no caractere **\d**. A exceção é quando a expressão regular é construída dinamicamente a partir de uma ou mais strings. Nesse caso, é preferível usar a **forma não literal**.*

Acesso a Propriedades

- Em JavaScript, a forma mais comum de acessar propriedades de um objeto é a **notação de ponto**, mas também é possível acessá-las com a **notação de colchetes** com uma string identificando o nome da propriedade.

```
var pessoa = { nome: "João" };  
console.log(pessoa.nome);  
console.log(pessoa["nome"]);
```

```
var array = [];  
array.push(11, 22, 33);  
array["push"](11, 22, 33);
```

- Embora a notação de ponto seja mais comum, a notação de colchetes pode ser útil quando é preciso decidir dinamicamente qual propriedade deverá ser acessada.
- Por exemplo, nesse exemplo, a notação de colchetes permite usar uma **variável** em vez de usar uma **string literal** (.push) para especificar a propriedade a ser acessada:

```
var array = [];  
var metodo = "push";  
array[metodo](11, 22, 33);
```

Identificando Tipos de Referência

- O operador **typeof** pode ser usado para identificar **funções**, como no exemplo a seguir. No entanto, ao tentar identificar outros tipos de referência usando **typeof**, o retorno será sempre a string **"object"**.

```
function retorna(valor) { return valor; }  
console.log(typeof retorna);
```

function

- Para isso, existe o operador **instanceof** que permite identificar outros tipos de referência. O operador recebe como parâmetros um **objeto** e o **tipo de referência**. Se o valor for uma instância do tipo especificado, o operador retornará **true**, caso contrário, **false**.

```
var itens = [];  
var objeto = {};  
function retorna(valor) { return valor; }  
console.log(itens instanceof Array);  
console.log(itens instanceof Object);  
console.log(objeto instanceof Object);  
console.log(objeto instanceof Array);  
console.log(retorna instanceof Function);  
console.log(retorna instanceof Object);
```

true
true
true
false
true
true

*Como todo tipo de referência herda do tipo **Object**, todo objeto também é uma instância de **Object**. É por isso que nesse exemplo **itens instanceof Object** e **retorna instanceof Object** retornam **true**.*

Tipos Wrapper Primitivos

- **Tipos de referência** especiais que permitem trabalhar com **valores primitivos** como se fossem objetos. Em JavaScript, os três principais tipos *wrapper* primitivos são: **String**, **Number** e **Boolean**.
- Os tipos *wrapper* são criados **internamente** sempre que strings, números ou booleanos são lidos. Com isso, é possível usar métodos destes tipos como, por exemplo, o método **substring** do tipo **String**.

```
var nome = "Pedro";  
var primeiraLetra = nome.charAt(0);  
console.log(primeiraLetra); // "P"
```

Nesse exemplo, a 1ª linha atribui um valor primitivo de string à variável **nome**. Note que a 2ª linha trata **nome** como um objeto, já que ele consegue chamar o método **charAt(0)**.

```
// O que a engine da JavaScript faz de fato:  
var nome = "Pedro";  
var temp = new String(nome);  
var primeiraLetra = temp.charAt(0);  
temp = null;  
console.log(primeiraLetra); // "P"
```

No exemplo acima, para usar a variável primitiva **nome** como um objeto, a *engine* da JavaScript cria uma instância temporária de **String (temp)** de modo que **charAt(0)** funcione. Na sequência, a instância temporária é modificada para **null**.

Tipos Wrapper Primitivos

- Embora **valores de referência** sejam criados automaticamente para **valores primitivos**, quando o tipo desses valores é verificado por meio de **instanceof**, o resultado não é o esperado:

```
var nome = "Pedro";  
var idade = 25;  
var ativo = false;  
console.log(nome instanceof String); // false  
console.log(idade instanceof Number); // false  
console.log(ativo instanceof Boolean); // false
```

O operador **instanceof** retorna **false** porque a instância temporária é criada internamente apenas quando o valor da variável é lido (p. ex.: **nome.charAt(0)**). Como **instanceof** não lê nada, a instância temporária não é criada. Logo, ele não reconhece que as variáveis são instâncias dos tipos **String**, **Number** e **Boolean**.

- Por isso, a forma correta para verificar o tipo de dado de **valores primitivos** é usando o operador **typeof**:

```
var nome = "Pedro";  
var idade = 25;  
var ativo = false;  
console.log(typeof nome); // string  
console.log(typeof idade); // number  
console.log(typeof ativo); // boolean
```

Tipos Wrapper Primitivos

- Vale ressaltar que o operador **typeof** não terá o comportamento esperado se os valores primitivos forem criados **explicitamente** por meio do operador **new**:

```
var nome = new String("Pedro");  
var idade = new Number(25);  
var ativo = new Boolean(false);  
console.log(typeof nome); // object  
console.log(typeof idade); // object  
console.log(typeof ativo); // object
```

*Instanciar explicitamente tipos wrapper primitivos pode gerar confusões ao usar o operador **typeof**, portanto, é uma prática que deve ser evitada.*

Classes

- Antes da **ES6**, a JavaScript não oferecia suporte formal para definição de **classes**, o que causava estranheza em muitos desenvolvedores acostumados com as linguagens orientadas a objetos tradicionais.
- Na ES6 foi introduzido o operador **class**, com o intuito de oferecer uma sintaxe mais intuitiva e concisa para declaração de objetos.

```
class Cubo {  
  constructor(baseA, baseB, altura) {  
    this.baseA = baseA;  
    this.baseB = baseB;  
    this.altura = altura;  
  }  
  calculaAreaBase() { return this.baseA * this.baseB; }  
  calculaVolume() { return this.baseA * this.baseB * this.altura; }  
};
```

As **propriedades de dados** devem ser declaradas dentro do método nativo **constructor()** e os **métodos** fora dele.

```
var cubo = new Cubo(3, 5, 12);  
console.log(cubo.baseA);      // 3  
console.log(cubo.baseB);      // 5  
console.log(cubo.altura);     // 12  
console.log(cubo.calculaAreaBase()); // 15  
console.log(cubo.calculaVolume());  // 180
```

Basicamente, as classes criadas na ES6 funcionam como uma “syntactic sugar” para declarar **tipos de referência personalizados**.

Referências

- SILVA, Maurício S. **JavaScript Essencial**. São Paulo: Editora Novatec, 2024.
- ZAKAS, Nicholas C. **Princípios de Orientação a Objetos em JavaScript**. São Paulo: Editora Novatec, 2014.